CS 24302 : COMPILER DESIGN LABORATORY

Lab Session 6 : Parser and Calculator for Arithmetic Expressions.

Scheduled : September 10 and 11, 2026

Faculty Team : Bikas Kanti Sarkar, Indrajit Mukherjee, Prashant Pranav, Mohd. Shah Fahad, Supreeti Kamilya and Supratim Biswas

Research Scholar : Swarna Aishwarya Twinkle

Recap :
1. Practice in writing CFG for features of C language.
2. Adding rules in the yacc script to manually draw the parse tree along with parsing.
3. Experiment with writing lex and yacc scripts for processing C code fragments

Objective : Generate a parser for C expressions using both ambiguous and unambiguous grammars. The numeric values are of integer and float type. Add semantic actions to the grammar to calculate the value of an expression and compare the calculated value with gcc. Extend the grammar to handle assignment statements.

Read the handout to understand how a lexer can effectively communicate with its parser in order to perform semantic actions along with parsing.

P1. The scripts, "**exp6.l**" and "**exp6.y**" use an unambiguous grammar to generate expressions in C involving 2 operators and parentheses {+, –, (,)}, and terminal {NUM} denoting integer values as operands.

E → E + T | T T → T * F | F F → (E) | NUM

Note that both the associative and precedence properties of operators are hard coded in the grammar and hence no special instructions are given in the yacc script. Observe how the actions against the rules use the "$-variables" of yacc to perform calculations. Read the handout to understand the association between the grammar symbols in a rule with the "$-variables".

(a) Generate a parser using the scripts given. Run the parser with the input in file, "**ex6inp.txt**". Also write a C program to find out the value of the input expression and validate the output of the C program with that generated by your parser.

P2. Examine the scripts of ambiguous grammar below that generates the same language as that of P1.

E → E + E | E * E | (E) | NUM

Write a yacc script that uses the above grammar to generate a parser and test its working over the input file, "**ex6inp.txt**". In case of difficulty, ask your faculty the way to specify operator properties for an ambiguous grammar in yacc.

P3. Extend the set of operators given in P2 with the addition of binary ops,{<, <=, ==, !=, &&, | |}. Design a parser that is consistent with C semantics(see Appendix for operator properties). A sample input and its desirable output is given as illustration. You have to augment both the scripts to solve this problem. The desired output for the input, 25 / 2 + 6 * 2 < 10 % 4 && 4 == 4 , in the file "**ex63inp.txt**" is shown below.

NUM Value is : 25 NUM Value is : 2 value of 25 / 2 is 12 NUM Value is : 6 NUM Value is : 2 value of 6 * 2 is 12 value of 12 + 12 is 24 NUM Value is : 10	NUM Value is : 4 value of 10 mod 4 is 2 value of 24 < 2 is 0 NUM Value is : 4 NUM Value is : 4 value of 4 == 4 is 1 value of 0 && 1 is 0

P4. All the problems defined above permitted only integer data type. We now wish to add floating point data type to the grammar. Since both integer and floating point numbers coexist in the grammar, both rules of the form, E → NUM | REAL will be present. The $$ variable for lhs E taken 2 different types depending on which token is sent by the scanner. In yacc there is a single $-variable associated with each grammar symbol (whether in lhs or in the rhs of a rule). Yacc permits a $-variable to be a union of 2 or more data types in order to meet such situations. Read the handout on lex-yacc communication for such details. Write lex and yacc scripts to generate a parser as per the specifications of this problem.

For the input, 2.0 / 3 - 5 / 2.5 * 6 + 12.2, in the file "ex64inp.txt", the desired output is shown below for your reference. Note that the rules for expression evaluation in C, when the operands of a binary operator are of different data types have to be used in the calculator.

Value is : 2.000000 Value is : 3.000000 value of 2.000000 / 3.000000 is 0.666667 Value is : 5.000000 Value is : 2.500000 value of 5.000000 / 2.500000 is 2.000000	Value is : 6.000000 value of 2.000000 * 6.000000 is 12.000000 value of 0.666667 - 12.000000 is -11.333333 Value is : 12.200000 value of -11.333333 + 12.200000 is 0.866667 Final Value of E : 0.866667

Attachments : "lex-yacc-communication.pdf", "exp6.l", "exp6.y", "ex6inp.txt", "ex63inp.txt", "ex64inp.txt"

End of the Experiment 6

Precedence	Associativity	Arity	Operator	Function of the operator
13	Left	binary	*, /, %	multiplicative operators
12	Left	binary	+, -	arithmetic operators
10	Left	binary	<, <=,	relational operators
9	Left	binary	==, !=	equality, inequality
5	Left	binary	&&	logical AND
4	Left	binary	||	logical OR
2	Right	binary	=, +=	assignment operators

End of the Experiment 6